

DSA STUDY BLUEPRINT SERIES

28. 2D Arrays in C++ Part1

Row-major and Column-major 2D Array Structures

Section 04 • C++ Core Data Structures & Algorithms

Core Concepts & Visual Blueprint

Theoretical models and memory architectures

Key Principles

- **Memory Model:** 2D arrays are stored in contiguous 1D memory locations.
- **Row-major formula:** $\text{index} = \text{row} * \text{cols} + \text{col}$.

Memory & Execution Blueprint

2D Grid Matrix

[1, 2]

[3, 4]

⇒

1D Linear Memory:

[1 | 2 | 3 | 4]

C++ Implementation Reference

Standard code layout and syntax

```
const int R = 3, C = 3;
int matrix[R][C] = {{1, 2, 3}, {4, 5, 6}, {7, 8, 9}};
for (int i = 0; i < R; i++) {
    for (int j = 0; j < C; j++) {
        cout << matrix[i][j] << " ";
    }
}
```

Algorithmic Complexity & Best Practices

Performance evaluations and critical guidelines

Performance Table

Aspect / Case	Complexity
Access Element	$O(1)$
Traverse Grid	$O(R * C)$

Key Practices & Gotchas

- **Important:** Row-major traversal is cache-friendly due to spatial locality.
- Verify boundary conditions (e.g. empty inputs, single elements).
- Optimize dynamic memory allocations to prevent leaks.

Dry Run & Step-by-Step Execution

Trace analysis on a sample input case

Execution Walkthrough

Let's dry run Row-Major index resolution mapping for element at 2D coordinate `[1][2]` (3x3 grid):

Row Index (i)	Col Index (j)	Formula: <code>index = i * cols + j</code>	Resolved 1D Offset
1	2	<code>index = 1 * 3 + 2</code>	Offset = 5 in linear memory block

Interview Variations & Optimization Pathways

Common follow-up problems and optimization strategies

Key Variations & Follow-up Questions

- **Row-Major Address Index mapping:** Flat memory locality makes sequential row scan speeds far faster than column-major scans.
- **Spiral Boundaries:** Keep strict checks inside individual boundary loop passes to avoid double print coordinates.
- **Related LeetCode Problems:** #54 Spiral Matrix, #74 Search a 2D Matrix, #48 Rotate Image.